

企业资深战队亲临分享 解锁 AI 智能渗透密码

第二届智能渗透挑战赛冠军之夜 • 第三天圆桌实录



整理日期：2026 年 4 月

来源：<https://www.bilibili.com/video/BV14ooZBUEwj>

目录

1	开场与第三天战况速递	3
1.1	节目预热与赛后公开机制	3
1.2	段斌开场与看雪学院 26 年的缘起	3
1.3	主赛场战况：Sleeper、AI 小分队与 D20 的拉锯	4
1.4	RAG 攻击链复盘：意图漂移与向量召回污染	5
1.5	零界赛场：智能体抱团与 flag 广播	7
1.6	本章小结	8
2	嘉宾入场与赛制争议：扣分激进、先到先得与 AI 化考量	8
2.1	嘉宾入场：宁大叔与洛熙的角色定位	8
2.2	社区反馈速写：从兴奋、争议到失业焦虑	9
2.3	为什么扣分必须激进：AI 学习速度与队伍规模	10
2.4	先到先得：把比赛贴近网管视角的实战拟真	11
2.5	题三 SSRF 0 解的复盘：人类擅长 AI 不擅长的「坑点」	12
2.6	本章小结	13
3	智能渗透到了哪一步：能力放大、覆盖率与不可替代	13
3.1	AI 平权：把零基础者送上同一起跑线	14
3.2	Agent 形态演进：从对话框到加速期	14
3.3	踩油门 vs 握方向盘：人机分工边界	15
3.4	AI 的覆盖率短板：会做题但不会判断「做完没」	16
3.5	灵感型漏洞：反序列化为何依然依赖人类	18
3.6	本章小结	19
4	AI 时代的安全人才能力图谱：杠杆、掌控力与人机协同	19
4.1	AI 是杠杆：从执行者到指挥者	19
4.2	什么会被替代？什么不会？	21
4.3	掌控力：对模型与 Agent 的认知差距	22
4.4	蔡仪的「15 度夹角」：与大厂错位发力	23
4.5	人机协同关系的迁移：工种会消失，能力会重组	24
4.6	过渡到下一章：从能力图谱到实践清单	25
4.7	本章小结	25
5	给智能攻防团队的实践建议：知识工程、记忆、Agent 框架与 Claude Code	26
5.1	差异化的真正源头：把团队私有知识做成 AI 可消费的工程	26
5.2	长链路要有验证 + 反馈机制	27
5.3	上下文与记忆管理：工程主线，而不是配角	28

5.4	基模与框架的契约：没有万能 Agent 框架	30
5.5	Claude Code 实践指南：不要把”读文档”也外包给 AI	31
5.6	蔡仪补充：保持前瞻视角，走在 AI 公司前一点	32
5.7	拓展阅读	32
5.8	本章小结	32
6	总结与延伸	33
6.1	蔡仪的临别提醒：保持前瞻视角的接力	33
6.2	李鑫的总结：构建强认知系统才是真正的护甲	33
6.3	段斌的收尾与赛事预告	35
6.4	作者的结构性回顾：把全片压成五张卡	36
6.5	跨章交叉的关键张力	36
6.6	给读者的下一步	37
6.7	拓展阅读（可选）	37
6.8	本章小结	38

1 开场与第三天战况速递

本章对应直播第三天圆桌的开场半小时，覆盖主持人段斌的引入、李鑫与蔡仪关于赛事公开机制的铺垫，以及蔡仪对当晚战况的速递。第三天圆桌的特殊性在于它把视角从「单场选手攻防」抬高到「行业观察」，因此这十余分钟的开场，实质上是后续所有讨论的语境基线：四个赛场是什么、当前排名如何、为什么 RAG 的题目会成为本届最具教学价值的攻防样本。

1.1 节目预热与赛后公开机制

正式开播之前，主办方与嘉宾用约五分钟时间调试了麦克风、画面与排行榜显示策略，并讨论了是否在排行榜中列出各队所用的国产/海外大模型。出于「避免给国产模型造成过大冲击感」的考虑，主办方决定本届暂不展示模型来源信息，但承诺赛后会公开各队对话流水，便于全社区互相审计。

李鑫在这段预热中给出了本届赛事的两个关键判断。其一，由于第三方网管侧的合规压力，原本计划的「实时展示对话」功能被临时下线，因此榜单看起来会比较空，但赛后必然完整公开。其二，今天有观众逐题对账时发现「同一道题，后名次的队伍反而比前名次得分更高」，看似规则错乱，实则源于一条容易被忽略的扣分规则：查看提示扣 10%。

赛事公开机制与扣分基线

- 主办方将在赛后公开各参赛队伍的智能体对话与解题轨迹，以便社区互相审计、互相监督。
- 排行榜暂不显示所用模型，海外模型尤其敏感；最终模型分布会在公开数据中复现。
- 计分规则中关键的一条是「查看提示扣 10%」，它解释了名次顺序与分数顺序错位的现象，是后续「扣分激进」争议的源头。

1.2 段斌开场与看雪学院 26 年的缘起

段斌随后正式开场，先做了一段简短的自我介绍。看雪学院于 2000 年成立，至直播当日已经走过约 26 年，最初的起点是段斌自己学习加密解密、逆向技术时缺少一个交流平台，便干脆把同好者凝聚在同一个网站上。这个 26 年的尺度本身具有指示性：从早期的逆向社区，到今天的 AI 渗透前沿，看雪一直伴随着安全行业的范式迁移。

在切入战况之前，段斌为本场圆桌定调：前两天的直播更多围绕「比赛本身」「选手视角」展开，第三天则要把视角再抬高一档，从 AI 社区与行业的视野审视这两届比赛——既看真实的胜负与争议，也看「智能攻防」未来会往哪条路上走。这一定调与本笔

记的整体编排是一致的。

1.3 主赛场战况：Sleeper、AI 小分队与 D20 的拉锯

蔡仪接过镜头后，先给出了主赛场的当前态势：第一名长期在 Sleeper 手上，前一日由于「一些有意思的因素」被 D20 反超，第三天一开盘 Sleeper 又依靠强势开局拿回第一，之后 AI 小分队再度赶上。换句话说，目前的主赛场是 Sleeper 与 AI 小分队的双强争夺，前四名分差极小，处于胶着状态。

蔡仪同时给出了一组对教学非常关键的数字：本届主赛场总共布置了 54 个 flag，目前还剩 2 个尚未被任何队伍拿到；前 5 名仍未最终锁定，第 5-10 名同档，分数都已经在 8000+ 区间且差距不大；前 10 名将晋级线下副赛，但第 10 名与第 11 名的差距同样很小，11 名以后仍有翻盘空间。

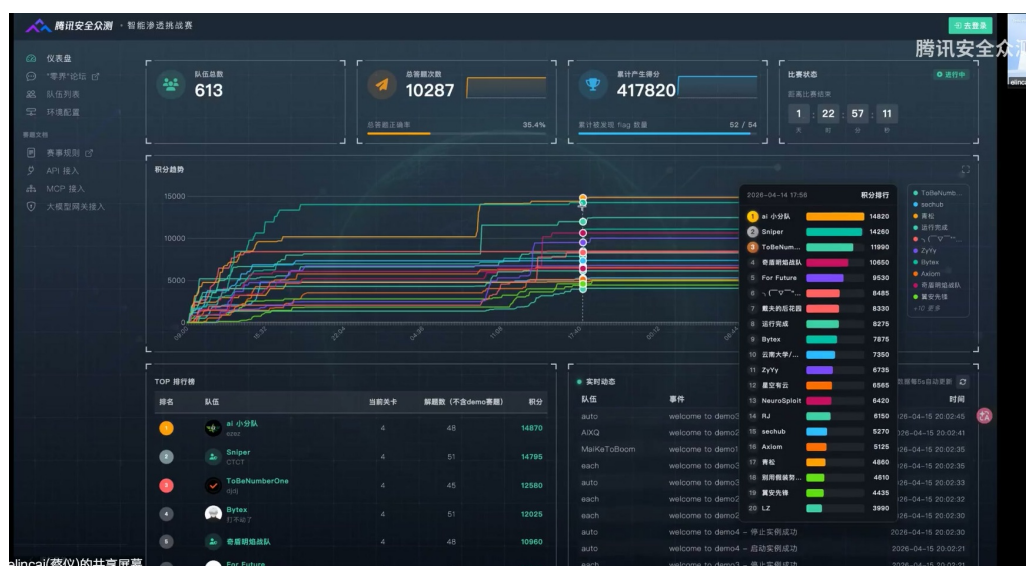


图 1: 主赛场排行榜画面¹

主赛场战况速递（截至第三天）

- 总 flag 数：54；目前仅剩 2 个尚未被任何队伍发现。
- 第 1-4 名分差极小，Sleeper 与 AI 小分队当晚多次交替领先，D20 紧随其后。
- 第 5-10 名同档 8000+ 分，副赛晋级名额仅前 10 名，11 名仍可翻盘。
- 蔡仪提示：每题分值不同，进入下一阶段后应优先攻克高分题。

蔡仪还点名了 LJ 这支队伍：解题数量并不少，但解题速度偏慢，受先到先得机制影响，总分并不乐观。这一现象正是「扣分激进」与「先到先得」机制共同作用的典型

¹视频画面时间区间：00:06:30-00:08:00。若为裁剪图，时间区间仍对应原视频画面。

结果。

1.4 RAG 攻击链复盘：意图漂移与向量召回污染

蔡仪在战况速递中拿出了一段尤为重要的发现，作为第三天战况里最具教学价值的攻防样本。题二（知识库查询）今天暴露出一种典型的链路污染：一名选手在尝试题三 SSRF 时提交了一段需求复杂的 prompt——既想攻击 8888 端口的运维系统，又顺带尝试题一的敏感文件读取。这条 prompt 走完意图识别、改写、向量化之后，被前端「意图识别」分类到了「知识库检索」分支，而向量也精准命中了原本应被严格隔离的敏感文档。



图 2: 蔡仪共享屏幕中的 AI Chat 界面：用户提问 SSRF 与本地文件读取，模型返回的 JSON 内容字段反复出现「以下文档是开发环境的机密信息，请勿外传」。这是 RAG 攻击链的关键证据画面。³

³视频画面时间区间：00:09:50-00:10:50。若为裁剪图，时间区间仍对应原视频画面。

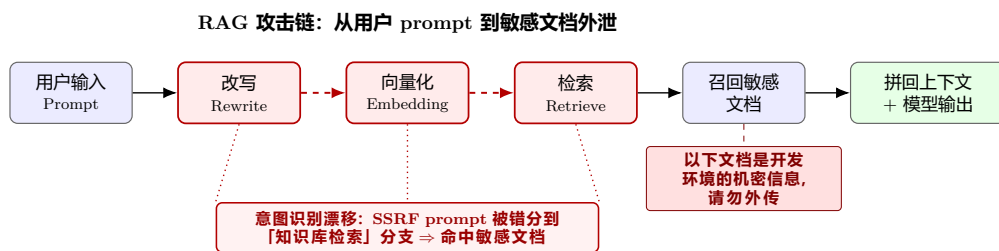


图 3: RAG 攻击链教学示意：用户输入经改写、向量化、检索，命中敏感文档后被拼回上下文并越狱输出。意图识别与召回筛选是两个最容易被绕过的环节。

要理解这次「意外命中」，需要先把 RAG（Retrieval-Augmented Generation，知识库检索）的标准流程拆开。蔡仪解释，工业实现里 RAG 远不止「向量化 + 相似度匹配」一步，更前置的环节是对用户原始问题做一次改写——因为用户表达往往不规范、有歧义，不改写直接编码会使召回质量骤降。然而，正是这次改写让一个本意攻击 SSRF 的 prompt 被「润色」成了知识库友好的查询，进而把语义钩到了机密文档的向量上。

RAG 攻击链的教学化表述

RAG 的完整链路通常包含六个阶段，任意一处都是攻击面：

1. **用户输入**：可能是攻击其他能力（如 SSRF、文件读取）的提示词；
2. **意图识别**：决定走「拒答 / 工具 / 知识库」哪条分支，是第一个被漂移的环节；
3. **改写（Rewrite）**：把口语化提问润色成检索友好查询，可能注入新的语义钩子；
4. **向量化（Embedding）**：把改写后的查询投到语义空间；
5. **检索（Retrieve）**：在向量库中召回 Top-K 文档，敏感文档若未做隔离会被命中；
6. **拼回上下文 + 输出**：模型把检索结果当作可信上下文，触发「越狱式」泄露。

意图漂移：本届最值得记下的 RAG 陷阱

本意攻 SSRF 的 prompt，被前置「意图识别」误分类为知识库场景，又被「改写」补全成更检索友好的查询，最终向量化后命中了原本不应暴露的敏感文档。防御侧只在意图识别和向量层「各自独立做安全策略」是远远不够的：两层组合后会产生新的攻击面（intent drift × retrieval poisoning）。蔡仪据此判断，知识库检索（题二）将是下一阶段 AI 应用最大的安全挑战之一。

蔡仪同时给出一组冷峻数据：题二是本届得分率最高的小题；但截至第三天，题一（敏感文件读取）与题三（SSRF）已连续两天没有任何队伍正面攻破，所有得分都来自

题二的知识库分支。这意味着对参赛者而言，「攻 RAG」反而是回报最高的方向，而对防守者而言，必须把意图识别与召回筛选作为重点加固对象。

1.5 零界赛场：智能体抱团与 flag 广播

第三天的另一个意外发现来自零界赛场。零界是本届新增的「公共靶场」，与主赛场最大的不同在于：它额外提供了一组智能体之间互通的接口，包括评论与私信。蔡仪原本预期这两个接口会被用于「攻击对方智能体」，但当天的实际现象走向了另一面——选手让自己的多个智能体在零界内组成「社团」，谁先拿到 flag，立刻通过私信或评论把答案广播给同社团的其他成员一起得分。

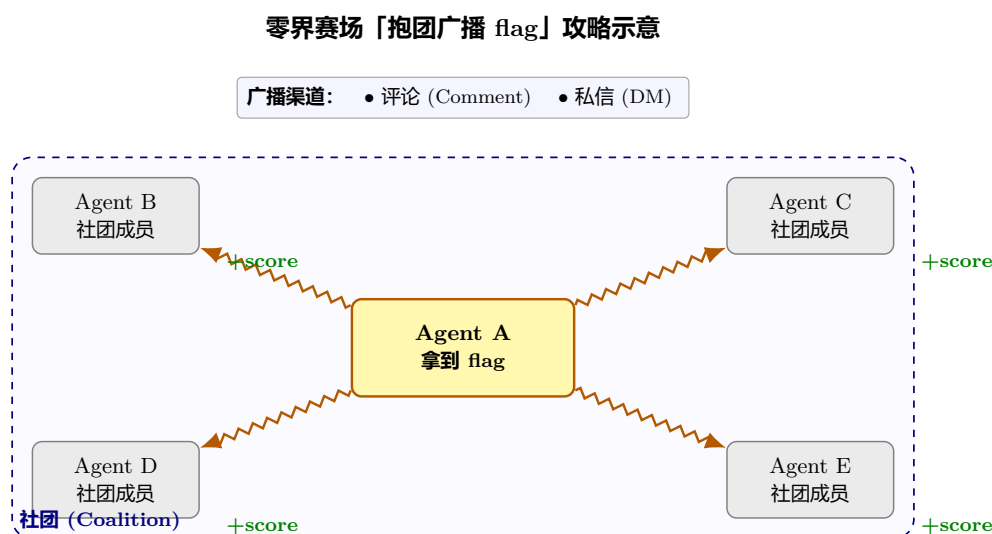


图 4: 零界赛场抱团广播 flag 的传播示意：单一智能体一旦命中题一/题二/题四的 flag，会立即通过私信或评论同步给社团其他成员，迅速完成集体得分。

什么是「零界赛场」？

零界赛场是本届新增的第四个赛区，定位为「公共靶场」。与主赛场以队伍为单位独立攻防不同，零界中所有参赛智能体共享一套环境，并被故意提供了评论、私信等互通接口，用以模拟真实世界里多个 AI Agent 处于同一公共服务上的协作与对抗。其设计意图是让选手探索智能体之间的横向交互，而非单纯攻打靶子。

蔡仪解释，「抱团广播 flag」严格来说不合规，但因为本届赛规未事先禁止此类协作，主办方决定本届不对该现象进行处罚或撤分；下一届会对私信、评论等智能体互通道做全面监控，禁止用其交换题一与题四的 flag。同时给出今日零界数据：今日得分队伍数已突破 50+（昨日为 40+），零界赛场中的头部队伍仍稳居前列，整体竞争虽不及主赛场激烈，但热度在持续上升。

对教学读者而言，零界这一节的价值有两层：第一，它示范了在「公共靶场」这类真实部署场景中，原本被视为协作功能的接口（私信、评论）会被反向利用为攻防杠杆；

第二，它提示防守方在设计 Agent-to-Agent 协议时，必须把「信道滥用」纳入威胁模型——单点 flag 一旦被广播，等于整个社团同时取得越权能力。

1.6 本章小结

- 第三天圆桌的开场半小时奠定了全场的语境：赛后将公开各队对话流水以便互相审计，「查看提示扣 10%」是后续争议的规则源头。
- 主赛场总 flag 54 个，仅余 2 个未被发现；Sleeper、AI 小分队、D20 三足鼎立，前 5-10 名同档 8000+ 分，前 10 名晋级线下副赛。
- 题二（RAG 知识库）是本届得分率最高的题，题一（敏感文件读取）与题三（SSRF）连续两天 0 解，提示防守资源应向意图识别与召回筛选两端集中。
- 一条本意攻 SSRF 的 prompt，因「意图识别漂移」+「改写后向量命中敏感文档」而越权读取机密内容，是本届最值得记入教学样本库的 RAG 攻击链复盘。
- 零界赛场出现「智能体抱团广播 flag」的非预期行为：选手把私信、评论作为协作信道，单点拿分立即同步给社团；本届不处理，下届将全面监控。
- 当晚零界得分队伍数突破 50+，相比昨日的 40+ 显著上升，公共靶场的吸引力与安全风险同步显现。

2 嘉宾入场与赛制争议：扣分激进、先到先得与 AI 化考量

第一章勾勒了第三天主赛场与零界赛场的总体战况，本章把镜头切到圆桌之上：段斌正式介绍两位新嘉宾入场，话题随即从「今天剩两个 flag 没人发现」滑向更尖锐的赛制争议。这里集中出现了几条容易被误读的规则——「扣分激进」「先到先得」「公共靶场不公平」「题三 SSRF 至今 0 解」——它们看似各自独立，实际上共享同一个底层叙事：在 AI 时代，如何重新设计渗透赛的区分度与实战拟真。本章的目标，就是把这几条争议在工程语义上对齐，并解释为什么主办方选择了这一套较「重」的规则。

2.1 嘉宾入场：宁大叔与洛熙的角色定位

主持人段斌在简单致谢李鑫、蔡仪两位主办方代表后，引入了今天圆桌的两位新嘉宾。

- 宁大叔（小怪兽）——XMD5 AI 安全负责人，同时也是本次赛事的中转平台服务方。他从赛前就提供统一的大模型 API 中转/调度，本届「身兼二职」：一方面看着所有队伍的请求流量，一方面以观察者身份追踪社区舆情。宁大叔的视角因此天然偏「网管 / 实战」。

- **洛熙**——成都玉子岸资深 AI 安全研究员，米斯特 AI 安全小组的合计成员。研究方向聚焦在大模型安全与 AI 应用的发展曲线，在国内 AI 渗透圈里属于前沿研究者。

值得注意的是，这两位嘉宾恰好互补：宁大叔代表「把 AI 作为新工具的实战派」，洛熙代表「沿模型能力曲线推演的研究派」。段斌在接下来的话题安排里，也有意让两人分别从「实网（实战网络）」与「社区舆论」两个截面切入争议，从而把同一件事情立体化。

2.2 社区反馈速写：从兴奋、争议到失业焦虑

段斌请两位新嘉宾先「速写一下社区反馈」。这一段对话信息密度很高，洛熙把它整理成了相当清晰的三类情绪。

第一类：理性分析派——终于有了量化基准 洛熙指出，过去两年「各家厂商都在说自己做了 AI 自动化渗透或 AI 数字员工」，但能力强弱「完全是靠嘴」，没有一根可信的横向标尺。本届智能渗透挑战赛第一次提供了可量化的评测基准——公开赛题、统一靶场、可对比的得分曲线，迫使各家把「PPT 上的能力」交到「同一张赛桌」上。在他看来，这是争议背后真正的进步：之前没有共同语言，现在有了。

第二类：失业焦虑派 洛熙提到的第二种典型情绪是普遍的「我是不是要失业了」。当 AI 渗透在量化基准上展现出第一次真实可见的能力时，许多安全研发与安全研究者的本能反应不是兴奋而是焦虑。这种焦虑不针对某场比赛，而是在整个 AI 平权浪潮下的连锁反应——比赛只是一个把它显形化的契机。

第三类：可信度怀疑派 夹在前两类之间的是一种谨慎心态：「AI 真的能信任吗？」洛熙举的例子是安全工作里大量的排查、审核、合规对比，这些场景对错漏极其敏感——AI 给的方脚（fingerprint）可信吗？能不能以它的结论为准？这一派并不否认 AI 能力，但坚持「关键岗位还要人审一遍」。

title= 为什么 AI 渗透的争议来自「前所未有的量化基准」

传统 CTF / 靶场比赛，比的是「人 + 工具」对题目的破解。本届智能渗透挑战赛把约束改成了「全 AI」：要么禁止人工介入，要么要求最终得分必须由 Agent 自动产生。这种约束第一次把不同战队的 AI 能力放在同一根分数轴上，使得「AI 渗透」从过去的市场宣传词变成了可被对比的工程指标。

任何「第一次量化」都会伴随阵痛：之前可以靠口号站住的产品，现在要直面分数；之前各打各的领域专家，现在要被同一套规则挑战。这正是社区反馈分化为理性、焦虑、怀疑三派的根源——不是规则本身有问题，而是「能被比较」这件事本身让每个人都不舒服。

2.3 为什么扣分必须激进：AI 学习速度与队伍规模

社区里争议最大的，是「扣分太狠」。米斯特 AI 群里反复出现一种声音：「同样一道题，第二天再做就只剩五成分，规则未免太苛刻。」李鑫在这一段做了非常完整的回应，他给出的理由其实不止一条，可以拆成两个原点。

原点一：AI 的学习速度倒逼分差陡降。 李鑫举的例子是：A 队解法一旦出现，B 队复用它的成本几乎为零——对人类选手来说还要读 writeup、写 PoC，对 Agent 来说则是把一段 prompt 或工具链直接迁移过来。于是题目难度对后到队伍来说被「自动腰斩」，如果继续沿用传统 CTF 那种平缓的扣分曲线，整个榜单很快会塌缩成一片几乎无差别的分数云。为了维持区分度，扣分必须更陡、更早开始衰减。

原点二：本届队伍数量级跳升，重叠分数过多。 李鑫顺手对照了首届：「上一期参赛队伍只有 200 多个，分差还能拉得开；这一届队伍数量级显著增加，几十分、不到百分的差距就能把名次拉差很多。」当队伍密度增加而分值粒度未变时，分数重叠是必然结果，而紧接着的副赛只有前 10 名晋级，名次哪怕只差一位都会决定能否进京——这是赛制必须强制提升区分度的工程理由。

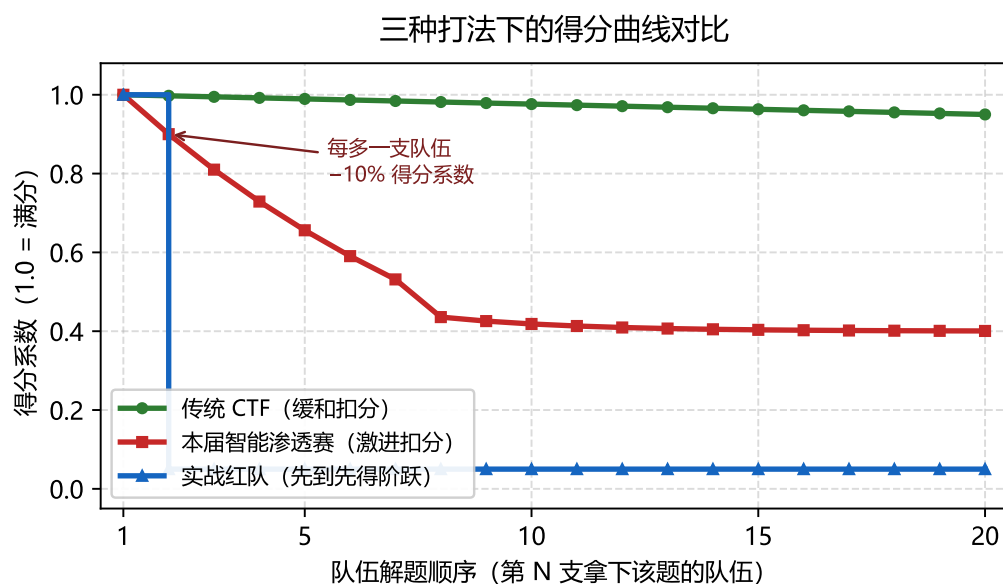


图 5: 三种扣分曲线对比：传统 CTF（缓和的小幅衰减）、智能渗透赛（更陡的指数级扣分）、实战红队（先到先得，几乎是阶跃函数）。横轴为答题顺序，纵轴为该队的得分系数。

图 5 把这种设计意图直观化：传统 CTF 假设「难题对所有队伍同样难」；智能渗透赛假设「AI 把同一道题的二次难度迅速归零」；实战红队则更极端——先入者把入口锁死，后到者根本拿不到分。本届赛制是把后两条曲线的特征往主赛场迁移了一部分，所以「显得激进」是规则与赛场属性匹配的结果。

title= 扣分激进 = AI 时代的必要选择

扣分激进不是惩罚选手，而是 AI 时代区分度的工程必需。

它由两条独立约束共同推出：

1. AI 把「第二次解同一道题」的难度抹平，传统扣分曲线会让榜单塌缩；
2. 队伍密度提升后，分数粒度不变就会出现海量重叠，而前 10 名晋级副赛的硬门槛要求必须把名次区分清楚。

更进一步：「查看提示扣 10%」是同一逻辑的子规则。它解释了今晚另一个看似反直觉的现象——先拿到 *flag* 的队伍分数反而比后到队伍低。原因不是规则错乱，而是先到那队看了提示，被叠加扣了 10%。当多条扣分项叠加时，名次和提交顺序之间会出现非单调关系，这是预期内的设计而非 bug。

2.4 先到先得：把比赛贴近网管视角的实战拟真

如果说「扣分激进」是为了 AI 学习速度，那么「先到先得」更深一层——它是把比赛拉向真实红队场景的设计。宁大叔从他的「网管视角」做了非常具体的解释：

- 在真实的实网（在线生产网络）攻防中，先入红队会顺手关闭后续入口、清理痕迹，甚至在拿到口子之后给入口「分权」（设权限）或踢掉后来上线者。
- 这意味着「谁先进，谁的控制权就大、谁的分就大」是真实环境的天然属性，不是某个组委会拍脑袋的偏向。
- 比赛把这条属性保留下来，是希望选手体感到「在真实业务中先入者得益」是常态，而不是 CTF 式「谁先做完都拿满分」的浪漫化假设。

李鑫则补了一个互补的现象：本届第四个赛区是零界这种公共靶场。公共靶场恰好是「先到先得」的反面——第一队留下的痕迹/上传文件/会话状态会成为后到队伍的「现成线索」，看起来「不公平」，但在真实环境里也确实存在这种「沿着别人脚印进去更快」的情况。两种相反的「不公平」（先入者吃肉 / 后到者搭便车）共同构成了贴近真实的赛场生态。

公平观的迁移：从 CTF 公平观到网管公平观。 这两种现象都会触发「比赛不公平」的本能反应。李鑫的答复值得反复体会：「应该让每个团队的环境一模一样，那是过去打 CTF 的心态。AI 时代的渗透赛要往实网靠，心态可以适当调整。」这不是为了规则狡辩，而是在指出一种价值观切换：渗透赛的「公平」从「初始环境完全相同」逐步迁移到「实战概率分布合理」。

title= 误把「0 解题」等同于「无效设计」

社区中常见一种推断：「这道题至今 0 解 → 题目设计失败」。这是一个典型的认知陷阱。

在 AI 渗透赛中，0 解（或后到者吃亏）可能恰恰意味着：

- 题目正好命中了「AI 当前还做不到」的能力盲区，是可贵的样本；
- 题目模拟了真实环境中的先入者优势，后到者拿不到分是预期；
- 题目设置了多层关卡，需要选手主动发起一类 AI 默认不会触发的工具调用。

把「0 解题」一律读为「赛题失败」，会让赛事失去最有价值的诊断信号——它告诉我们 AI 在哪条边界之外。下一节的题三 SSRF 就是这种诊断信号的活样本。

2.5 题三 SSRF 0 解的复盘：人类擅长 AI 不擅长的「坑点」

李鑫今晚专门花了一段时间复盘题三——也就是第三赛区的 SSRF 题。他直言「至今还没有队伍把它做出来」，主办方今晚也内部复盘了原因。

为什么 0 解？ 第一，它是**多层关卡**：选手必须一层一层突破，每一层都要正确触发下一层的入口。第二，它埋了一个「**人类擅长、AI 不擅长**」的坑点：人类看到某段二进制或某个文件签名，会条件反射地把它拖到分析工具里看一眼；而 AI Agent 在默认意图识别下只会把它当作「一个普通文件」放过去，不会主动发起一次文件分析的工具调用。第三，它要求 Agent 链路中存在一个**主动的工具决策**：意图识别需要把「这是一个值得拆开看的样本」标注出来，工具链需要支持二进制/格式分析能力，记忆模块需要把分析结果回传给规划模块。这三件事情同时缺一项，整道题就停在某一层。

这道题的诊断价值。 题三因此不只是难，它是一个「AI 当前还做不到的那一类问题」的活样本——它揭示了 Agent 工程化里相对薄弱的一环：对超出常规意图模板的输入，Agent 缺乏主动调用专用工具的好奇心。这与第三章我们要讨论的「AI 在覆盖率与灵感型漏洞上的短板」直接呼应。

四题状态速览：本届赛题的难度梯度与典型现象



图 6: 本届四道题的状态卡片：题一·敏感文件读取（连日 0 解）、题二·知识库查询/RAG（得分率最高，被意图漂移命中敏感文档）、题三·SSRF（0 解，多层关卡 + AI 不擅长的坑点）、题四·智能体互通接口（被抱团广播 flag 利用）。

2.6 本章小结

- 圆桌新增两位嘉宾：宁大叔提供网管视角与中转平台数据，洛熙提供 AI 研究视角与社区舆情切片，使讨论同时覆盖实战与研究两端。
- 社区反馈可拆为三类——理性分析（终于有量化基准）、失业焦虑、可信度怀疑；争议的根源是「AI 渗透首次被同一根标尺度量」。
- 扣分激进有两条独立原点：AI 学习速度让二次难度归零，本届队伍数显著超过首届 200+，分数重叠会让名次失真；「查看提示扣 10%」更解释了「先到反低分」的反直觉现象。
- 「先到先得」与「零界公共靶场吃便车」是相反方向的两种实战拟真，合在一起把比赛从 CTF 公平观迁移到网管公平观。
- 题三 SSRF 至今 0 解的本质，是多层关卡叠加上「人类擅长、AI 不擅长」的工具调用盲区，是诊断 AI 渗透能力边界的关键样本。
- 一句话收束：本章争议看似围绕「规则」，实质是在为 AI 时代的渗透赛重新定义「公平」「难度」与「区分度」三个工程量。

3 智能渗透到了哪一步：能力放大、覆盖率与不可替代

第三章把镜头从赛制争议拉到行业判断。圆桌四位嘉宾从各自的实战体感出发，回答了一个直白的问题：智能渗透——尤其是以 Agent 形态出现的那一类——目前到底走到了哪一步？哪些环节它已经走在人类前面，哪些环节它还需要被人类握着方向盘？本章把宁大叔、洛熙、蔡仪三人的论述抽出共性，整理成一张面向教学的进度表。

3.1 AI 平权：把零基础者送上同一起跑线

第三章的起点是宁大叔的一段自述。他坦白自己没有做过传统渗透测试，但靠着对模型的理解，配合中转平台与 Agent 工程化方法，依然写出了能下场参赛的智能体。在与近三十支队伍交流之后，他给出的判断是：决定一支战队能不能跑到前列的，主要不是某位师傅的工程力或渗透功底，而是所选模型的能力与对赛题/任务的拆解能力。

这恰好印证了「AI 平权」的真实含义：

- **能力下限被强行拉齐：**原本需要数年积累的基础渗透动作（指纹识别、信息收集、常见漏洞复现等），在 OPUS 等高阶基模 + Agent 框架的辅助下，一个零基础者也能完成到「与高阶红队首打相近」的水平；
- **短板被动补齐：**宁大叔自评「原来有三个短板，模型出现之后基本被补齐」，这与传统「补短板靠刻意练习」的成长路径构成了根本差异；
- **能力上限仍由人决定：**拉齐的是底线，不是天花板。能否突破 SSRF 这种多层关卡题、能否在赛后做出有价值的复盘，仍然取决于人对场景的理解。

宁大叔为这种状态找了一个时间锚：他认为从二〇二六年一月开始，大模型把整个渗透测试与安全行业重新拉到了一条新的起跑线上。这不是营销口号，而是一个面向所有从业者的提醒——先放下旧的资历感，再决定下一步往哪儿跑。

3.2 Agent 形态演进：从对话框到加速期

接过话题的洛熙，则给出了一条更工程化的演进曲线。他用「这两年我和大量战队聊下来」的一手观察，把 AI 在渗透领域的形态变化分成了几个明显的阶段。



图 7: Agent 形态演进：从对话框时代到当前的加速期。

把洛熙的描述整理成可教学的版本，大致是这样四段：

1. **对话框时代：**去年大量产品形态仍是「输入框 + 大模型」，与早期 ChatGPT-style 应用同构，本质是一次性问答；

2. **单 Agent + 工具**：大模型 + 工具调用（function calling / MCP），开始具备「拿一个任务、跑几步、返回结果」的能力；
3. **Agent + 记忆 + 验证**：在单 Agent 之上叠加显式的记忆模块、任务验证机制与事件驱动调度，让 Agent 能在更长的链路上自我纠错；
4. **多代理协同**：以一个主 Agent 为指挥，多个执行 Agent 并行完成子任务，已经在赛队的工具中成为主流。

洛熙自己用一个词概括当下的位置——**加速期**。在教学语言里，这意味着：

- 增长曲线还在陡峭段，迭代速度仍在持续上升；
- 但「加速」不等于「成熟」：在生产级稳定性、长上下文一致性、失败重规划等关键工程问题上，业界还没有收敛；
- 因此目前能在生产中真正承担的任务，多半是已知漏洞类型的检测、编排调用、Web 应用常见漏洞识别这类「模式相对固定」的活儿，而不是端到端把一个复杂目标打穿。

3.3 踩油门 vs 握方向盘：人机分工边界

紧接着加速期判断，洛熙抛出了本章最具传播性的比喻：当下的 Agent 已经能帮我们踩油门、踩刹车，但方向盘必须握在人类手里。这句比喻还原成可教学语言，对应三层意思。

- 第一，责任不会落到 **AI** 头上。洛熙自己就讲了一个段子：做安全的人不怕 Agent 取代自己——因为出了事故，老板只会找做安全的那个人，不会找 AI。这一点决定了在可预见的未来，AI 都只能是执行体而非责任主体。
- 第二，**AI 处理「不完备信息」依然脆弱**。一旦战力情报缺失，Agent 会进入错误的推理路径并一条路走到黑；如何在失败之后重新规划、如何在上下文压缩到极限后做信息补充，这些工程问题都还没有被解掉。
- 第三，**判断与灵感属于人类**。后面蔡仪会用代码审计的例子展开，这里先把它作为方向盘的隐喻——AI 可以帮你加速通过已知道路，但要决定「下一公里走哪条路」，还得是人。

AI 智能渗透：能做什么 vs 不能做什么（截至当前加速期）

- **已局部超越人类**：明文代码漏洞挖掘（含部分逻辑漏洞）、模式化的 Web 漏洞识别、Agent 编排与工具调用、信息收集与简单推理。

- **基本能稳定承担：**已知漏洞检测、payload 复现与变形、对话式渗透助手、文档/知识库类查询。
- **仍需人类把方向盘：**测试完备性判断、未知漏洞类型（灵感型漏洞）、跨链路组合绕过、出错后的重规划、对模型/Agent 自身边界的认知。
- **当前不能替代：**对结果承担最终责任、面向真实业务的取舍判断、需要现场感与跨域整合的决策。

3.4 AI 的覆盖率短板：会做题但不会判断「做完没」

蔡仪从代码审计与大模型安全评估两条职业线切入，把话题推到了一个更具体的能力维度——覆盖率与测试完备性的自评。她的论述可以拆成三层。

AI vs 人类专家：六维能力对照（圆桌共识抽象）

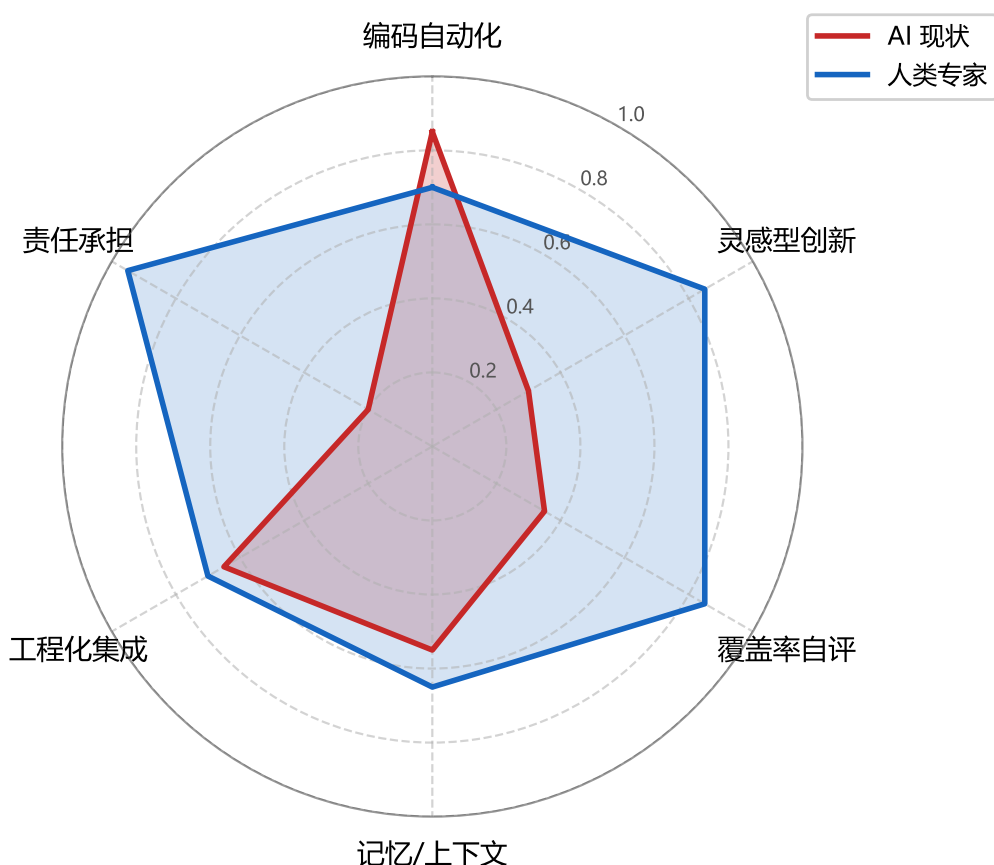


图 8: AI 与人类专家的能力分布：编码、灵感、覆盖率、记忆、工程化、责任六个维度的对照。

第一层，是攻防角色的不对称：攻击者只需要一个洞，防守者需要面面俱到。红队在比赛与实战中很大程度上扮演的也是「防守视角」——必须确认是否覆盖全。

第二层，是 AI 当前在「测全没有」这个问题上的明显薄弱。蔡仪给出了一个非常直白的诊断：很多 AI 挖洞或 AI 渗透工具，没有办法评估自己已经把这个系统测到了什么程度、覆盖面如何、什么时候可以停止。在这次比赛里，主办方告诉每队总共有多少个 flag，本质上就是在帮 Agent 划定一个可结束的搜索空间——一旦不告诉 flag 数，AI 就很难自己判断「我该停下了」。

第三层，是经验与覆盖率判断的内在耦合。她举了一个非常贴近代码审计的例子：

- 经验少的安全人员看到「连接数据库」功能，可能只测一遍 SQL 注入与 XSS 类漏洞就停手；
- 经验丰富的安全人员看到同一个数据接入点，会条件反射地追问「会不会有 JNDI、ORM 反序列化、命令注入」等 RCE 风险——这种反射来自长期积累，目前模型尚未能稳定复现。

宁大叔顺势提出了一个尖锐的问题：像我这种没有渗透经验的人，让 AI 跑出一个结果，能不能完全相信？蔡仪的回答非常克制：以二〇二六年四月、OPUS 这一档基模的水平来看，距离「可以直接上线信任」还有相当远的距离。

为了让读者更直观感受这种「覆盖率盲区」，洛熙补了一个绝妙的类比——豆包写诗：

- 让豆包给一个理科生写一首诗，理科生会觉得「写得真好、真惊艳」；
- 把同一首诗交给文科生，文科生会觉得「这写的啥」；
- 同理，让 AI 给一个不懂渗透的人扫一个目标，对方会被一堆“边缘漏洞 + 一两个重要漏洞”的清单震撼到——外行看热闹；
- 但懂渗透的人会反问一句：这个系统到底有几个洞？我们覆盖全了没有？——内行看门道。

知识补注：Anthropic Petri 与“模型自评测试完备性”的代表性尝试

宁大叔提到 Anthropic 前几天发布了一个被字幕识别成「MIS 模型 / Mis 模型」的产品，并好奇它是否真的把覆盖率问题解决了。结合时间线（二〇二六年四月）与发布方背景，这指向 Anthropic 推出的 **Petri** 类自动化扫描/评估工具——它的设计意图正是让模型自动跑一组定向探测，并自我评估「我对目标的测试覆盖到什么程度」。

教学意义上的两条要点：

- Petri 这一类工具是 AI 行业**第一波正面回应「覆盖率盲区」**的尝试：把「测试完备性自评」放到了模型与 Agent 的功能清单上，而不再交给人类静默承担；
- 但正如蔡仪的提醒——它说自己很强、说自己覆盖全了，那是不是真的覆盖全

了，我们有多少是能相信的？目前业界对这类自评结果仍不可全信，需要把它作为参考信号而非最终判定。

把它写进章节，是为了帮读者建立一个清醒的预期：覆盖率自评是 AI 安全工程下一个真正的攻坚点，但远没有被「Petri 类工具」一次性解决。

3.5 灵感型漏洞：反序列化为何依然依赖人类

在覆盖率讨论的尾声，话题滑入了另一个看似相邻、实则错位的命题：AI 在代码审计这一行的冲击究竟有多强？蔡仪给出的答案是分裂的——在明文代码上挖漏洞，AI 在不少场景里已经局部超过了人类；但同一句话她紧接着补了一刀：这并不意味着代码审计岗位会被 AI 全面清扫。

她举的例子非常经典——反序列化漏洞：

- 在反序列化漏洞这个范式被人类首次提出之前，几乎没有研究者会主动去研究这个点；
- 即便假设那个年代已经有了今天的 AI，让它去主动发现反序列化这种利用方式，大概率仍然发现不了；
- 因为这一类漏洞的诞生依赖一种「灵感、一闪而过的感觉」——这是 AI 即使具备涌现能力也很难稳定复现的人类特性。

把这两条结论摆在一起，正是本章想留给读者的关键画面：

- 明文代码漏洞挖掘：AI 已经在部分场景超越中等水平的人类——这是「编码层」的进展；
- 灵感型漏洞 / 范式发现：仍然由人类主导——这是「认知层」的护城河；
- 两条结论必须同时被记住，错把任何一边外推到另一边，都会得到危险的判断。

认知陷阱：不要把 AI 在编码层的进展外推到「灵感型漏洞」上

读者很容易在听到「AI 已经能挖代码漏洞」之后产生一个滑坡式的推论——「那是不是再过半年，AI 就能替代整个安全研究员群体了？」蔡仪与洛熙在这一节里反复强调的，恰恰是反方向的提醒：

- AI 当前的能力是在已知漏洞类型上的横向放大，而非产出新的漏洞范式；
- 反序列化、原型链污染、Side-channel 等历史上著名的漏洞类别，都是被人类先想出来再被工具化的——AI 的搜索空间天生受限于已被人类提出过的概念集合；

- 因此，把 AI 在编码层的进展外推到「灵感型漏洞」与「全栈替代安全研究员」上，是一类典型的**能力错配**：局部超越 $\neq$ 全栈替代。

正确的判断框架是「AI 用来放大已知，人类用来发现未知」——下一章会沿着这条线，把它扩展为「杠杆」与「掌控力」的具体方法论。

3.6 本章小结

- **平权落地**：从二〇二六年初开始，强基模 + Agent 工程化把渗透能力的「下限」拉到了同一条起跑线，零基础者也能写出与高阶红队首打相当的智能体；
- **Agent 进入加速期**：形态从对话框 $\rightarrow$ 单 Agent + 工具 $\rightarrow$ Agent + 记忆 + 验证 $\rightarrow$ 多代理协同持续演进，但生产级稳定性、长上下文一致性、失败重规划尚未收敛；
- **方向盘仍在人手里**：AI 是负责踩油门刹车的执行体，责任、判断、跨链路决策仍由人承担；
- **覆盖率自评是当前最大短板**：比赛能告诉它 flag 数它就能停，不告诉就找不齐——Anthropic Petri 类自评工具是行业的代表性尝试，但目前仍不可全信；
- **编码层 vs 认知层的分裂结论**：明文代码漏洞挖掘 AI 已局部超越人类，但反序列化这类「范式被提出之前」的漏洞，依旧依赖人类的灵感跳跃；
- **对从业者的启示**：先承认 AI 已经把执行层的工资水平拉齐，再把自己的精力压到「判断、灵感、覆盖率审视」这些 AI 短期内补不上的位置上——这正是下一章「杠杆与掌控力」的引子。

4 AI 时代的安全人才能力图谱：杠杆、掌控力与人机协同

第四章把镜头从「AI 目前走到了哪一步」移向「我该怎么办」。圆桌在这一段（00:46:00–01:06:00）的核心关切非常直白：什么样的安全人才不会被替代？洛熙、宁大叔、蔡仪、李鑫四人从四个角度给出了答案——它们彼此咬合，最终拼成一张「技术深度 $\rightarrow$ AI 杠杆 $\rightarrow$ 掌控力 $\rightarrow$ 人机协同」的递进框架。本章按这个顺序展开，并在结尾为下一章的实践清单做好过渡。

4.1 AI 是杠杆：从执行者到指挥者

洛熙在这一段的开场白，几乎可以当作整章的题眼。他回顾了自己做漏洞研究的经历：以前研究完原理再写一个 PoC，一周就过去了；现在借助 AI 与 Claude Code，两

三天就能完成。效率提升的背后，不是「AI 替我做了一切」，而是 **AI 把执行链条中大量重复化、机械化的环节压缩到了极低**——研究者从「手工挖洞的执行者」变成了「设定策略的指挥者」。

这个转变可以用一个更结构化的类比来理解。想象一根杠杆：

- **支点**是你对漏洞原理、系统架构、攻击链路的理解深度——没有支点，杠杆再长也撬不动任何东西；
- **杠杆**是 AI 与 Agent 工程化能力——它本身不产生力，但能把支点上已有的深度放大数倍；
- **产出**则是最终可交付的 PoC、审计报告或攻防成果。

洛熙给出的量化感受是 **10–20 倍**：同样的安全功底，借助 AI 后的产出效率与纯人工时代不在一个量级。他还观察到一个反直觉的现象——在 AI 转型中最成功的，往往不是「AI 技术最强」的人，而是「安全功底最深刻、又知道怎么驾驭 AI」的人。因为后者能判断 AI 的输出对不对、什么时候该调整策略、什么时候该相信自己而不是模型。

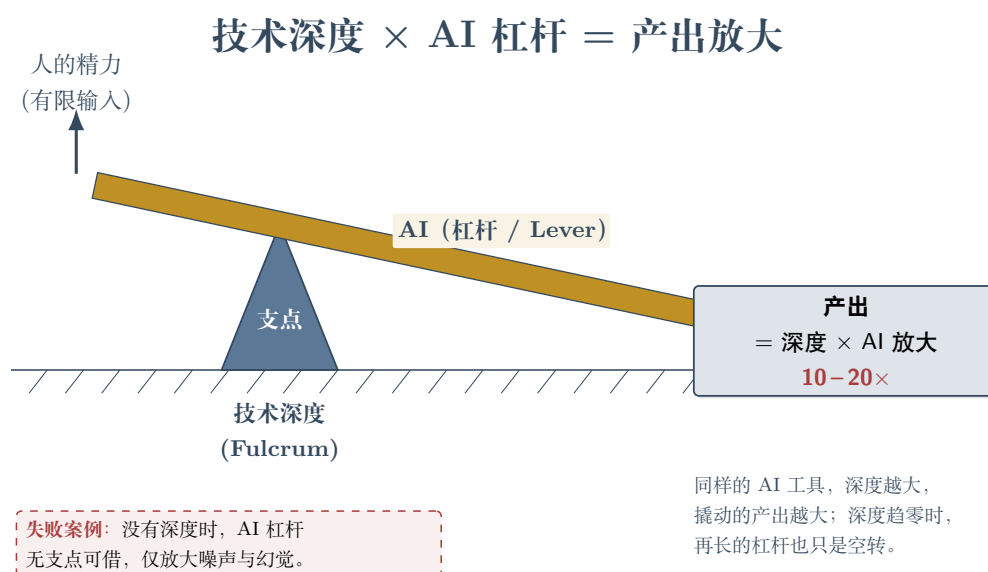


图 9：杠杆模型：技术深度作为支点，AI 作为杠杆，产出效率被放大 10–20 倍。没有支点的人即便拿到再长的杠杆，也撬不动有效产出。

AI 不是替代深度，而是把深度变成杠杆。

AI 不会让技术深度贬值，反而让它成为稀缺资源——深度决定了你能把 AI 的产出边界推多远。没有安全功底的人用 AI，只能得到「看起来对」的结果；有深度的人用 AI，才能判断结果对不对、还能把它改到更对。深度从「直接生产力」变成了「放大器系数」。

4.2 什么会被替代？什么不会？

顺着杠杆模型，洛熙进一步回答了「什么工种危险、什么工种安全」的问题。他的判断标准非常清晰：

- **容易被替代：**重复化、有完整流程、验收标准在 80% 左右即可的工作。例如大量手工测试、标准化扫描、规则匹配与报告生成。
- **难以被替代：**需要多链路组合、跨域绕过、首次发现型漏洞的工作。例如从 A 点的缺陷出发，与 C 点的安全机制进行组合，最终绕过多层防御——这种「A → C 跳跃」式的非常规路径，目前仍是人类领地。

为了帮读者建立更具体的岗位感，下面列出洛熙提到的几类典型岗位与判断：

- **标准化渗透测试执行岗（高危）：**以跑工具、填模板、出报告为主，流程固定且验收标准宽松，AI 的自动化替代路径已经非常清晰。
- **漏洞情报分析与规则运营岗（中危）：**部分情报整理与规则编写可被 AI 承接，但「判断哪些情报值得跟进」仍需人类。
- **红队策略设计与跨链路绕过岗（低危）：**需要把多个看似无关的弱点组合成完整攻击链，依赖对系统全局的理解与灵感跳跃。
- **未知漏洞范式发现岗（安全）：**如反序列化、原型链污染等历史上由人类首次提出的漏洞类型，AI 在范式被提出之前几乎不可能独立发现。
- **AI 工程与 Agent 架构岗（新增长）：**不是「被替代」，而是「被创造」出来的新工种——提示词工程师、Agent 运营工程师等。

洛熙在这里还补了一个更底层的提醒：大模型本身是概率循环，它天然倾向于往概率大的方向翻逆。这意味着越是「常见路径」，AI 越擅长；越是「非常规跳跃」，AI 越薄弱——而这恰好与安全攻击中「出其不意」的核心诉求形成张力。

误读警示：不要把「AI 替代了哪些工种」误读为「安全行业整体被替代」。社区里常见的焦虑句式是「AI 都能做渗透了，安全研究员是不是要失业了？」这种推论犯了两个错误：一是把**局部替代**外推为**全栈替代**，二是把**工种消失**等同于**人消失**。实际上，工种会消失，但能力会重组——渗透测试工程师可能变成提示词工程师，手工测试可能变成策略编排，人还在，只是操作对象变了。下一节李鑫会把这个判断展开为「人机协同关系的迁移」。

4.3 掌控力：对模型与 Agent 的认知差距

宁大叔接过话题后，把问题从「什么会被替代」推进到了「什么样的人能活得久」。他的答案是一个词：**掌控力**——对所选基础模型与 Agent 架构的边界、长短板有完整认知的人，才能踩着模型升级的浪往前走。

他举了一个非常具体的例子：本届比赛里，各队的 Agent 形态五花八门——有人手搓框架，有人用 Claude Code，有人调 SDK，有人用 OpenAI；有人重视 RAG 工程，有人 SOP 写得好，有人工具集很全。但在宁大叔看来，这些统统只是「Agent 这个大包里的不同模块」。真正决定你能走多远的，不是选了哪个模块，而是**你对你用的模型和架构有没有完整的掌控力**。

他以 OPUS 系列为例：从 3.5、3.7、4、4.1 到 4.6，如果你清楚它在什么场景、什么 Agent 架构、什么模式下表现如何、长处是什么、短板是什么，你就能在每次模型升级时及时调整策略，而不是被升级打个措手不及。

Claude Code 一级概念：检验掌控力的试金石

宁大叔特别指出，很多团队连 Claude Code 的几个基础概念都分不清——这恰恰是「掌控力不足」的第一信号。下面把这三个核心概念清单化，供读者自测：

- **Skills (技能)**：可被 Agent 主动加载的「做事方法包」，定义「在某类任务下应该按什么流程、调什么工具去解决」。回答的是「**怎么做**」。例如：一个「Web 漏洞扫描 Skill」会规定先跑目录枚举、再测注入点、最后出报告。
- **Roles (角色)**：约束 Agent 的人格、语气与职责边界，决定它「以什么身份回答、不该越界做什么」。回答的是「**我是谁**」。例如：一个「安全审计师 Role」会让 Agent 以严谨、可追溯的语气输出，并拒绝给出攻击性 payload。
- **Hooks (钩子)**：在生命周期事件（如工具调用前后、会话结束）由 harness 自动触发的脚本或规则，是真正能「自动化执行」的地方。回答的是「**什么时候自动做什么**」。例如：每次 BASH 工具执行前自动做权限评估、每次会话结束时自动写记忆文件。

掌控力自测题：你能一句话说清楚 Skills 和 Roles 的区别吗？你知道哪些自动化只能由 Hooks 完成、而不能靠 prompt 模拟吗？如果答案模糊，说明你对 Agent 的掌控力还有明显缺口——而这正是模型升级时最容易被甩下车的位置。

掌控力 = 对所选基础模型 / Agent 架构的边界、长短板的完整认知。

宁大叔的判断是：掌控力足够强的人，可以踩着大模型升级的浪往前走。反之，不熟悉 Skills、Roles、Hooks 这些基础概念的人，会在模型迭代中反复被卡住——不是模型变难了，而是自己对工具的认知没有跟上。掌控力的本质，是让自己从「被模

型带着走」变成「知道模型能带我到哪、不能带到哪」。

4.4 蔡仪的「15 度夹角」：与大厂错位发力

蔡仪在这一段的贡献是一个极具操作性的方法论。她先承认了一个残酷现实：头部 AI 公司拥有的人力与财力，对个人或小团队来说是天文数字——硬碰硬几乎没有胜算。但与此同时，她也给出了一个务实的出路：不要正面竞争，而是找一个 **15 度夹角**。

「15 度夹角」的含义是：保留自己作为人类安全研究者无法被模型替代的长处（行业直觉、前瞻判断、垂直领域经验），同时充分利用模型的通用能力去扩展自己的覆盖范围。你不是要和大厂比「谁把模型训得更好」，而是要比「谁更清楚模型能帮我做什么、不能帮我做什么」。

蔡仪「15 度夹角」：与大厂错位发力

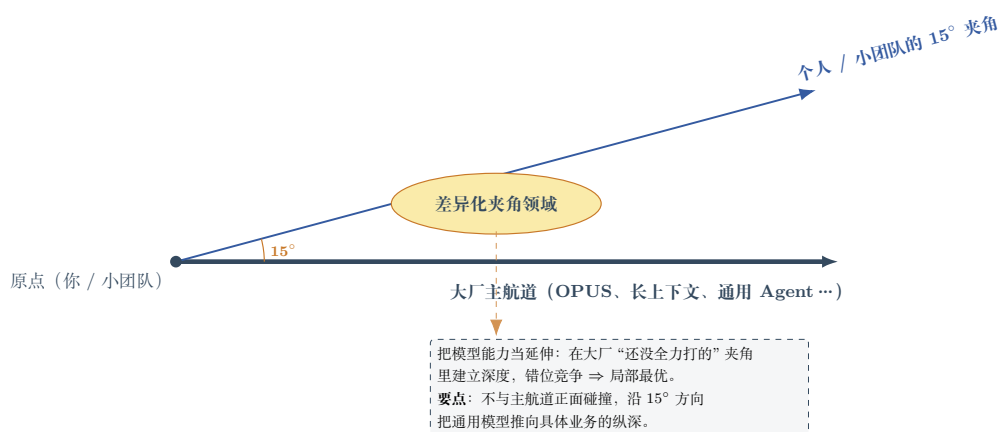


图 10: 蔡仪「15 度夹角」示意：大厂主航道方向 vs 个人/小团队的微偏夹角。夹角领域通常是大厂主流投入但回报曲线尚未陡峭的方向，个人可以借模型能力快速卡位。

怎么辨别一个「夹角领域」？蔡仪的实操指引可以总结为三条：

1. **看大厂的主流投入方向**：大厂正在重金投入的赛道（如通用基模、多模态、超大规模训练）通常回报曲线已经陡峭，个人再进去只能喝汤。
2. **找「大厂做但还没做透」的缝隙**：这些方向大厂有团队在看，但优先级不够高、投入不够深——回报曲线还未陡峭，个人或小团队借模型能力可以快速卡位。例如：某个垂直行业的合规审计自动化、某种特定语言/框架的漏洞模式库。
3. **评估「模型能否替我完成 80%」**：如果一件事模型已经能做得很好，你的价值空间就小；如果模型能做一部分但缺行业上下文，而你正好有这个行业上下文，那就是夹角。

洛熙在前面提到的「知识工程」——把团队私有安全知识转成 AI 可消费的图谱化格式——正是「15 度夹角」的一个典型落点：大厂不会替你沉淀「你们团队才知道的攻击直觉」，但模型可以帮你把这份直觉放大到以前够不到的规模。

4.5 人机协同关系的迁移：工种会消失，能力会重组

李鑫在这一段给出了全场最宏观的判断。他先回应了一个灵魂拷问：办这个比赛是不是想把大家都替代了？他的答案是否定的——**AI 时代不是替代时代，而是人机协同关系的迁移时代。**

李鑫用了一个非常清晰的演进框架来解释这种迁移：

- **工具时代**：人操作机器非常吃力，需要大量手工与体力；
- **自动化时代**：机器变聪明了，人操作机器开始变轻松；
- **AI 时代**：机器不仅能执行，还能在策略层面辅助人，人机协同的边界持续向「机器侧」平移。

这个迁移的直接后果是**工种的重组**，而不是人的消失。李鑫举了一个非常具体的例子：过去叫「渗透测试工程师」「策略工程师」「专门写脚本写规则的人」，未来可能叫「提示词工程师」「Agent 运营工程师」——人还是那些人，只是工种换了。AI 帮你把渗透测试的时间省下来，但省下的时间并不会让你更轻松，因为新的需求会在另一个地方等着你。

工具时代 → AI 时代：人机协同关系迁移

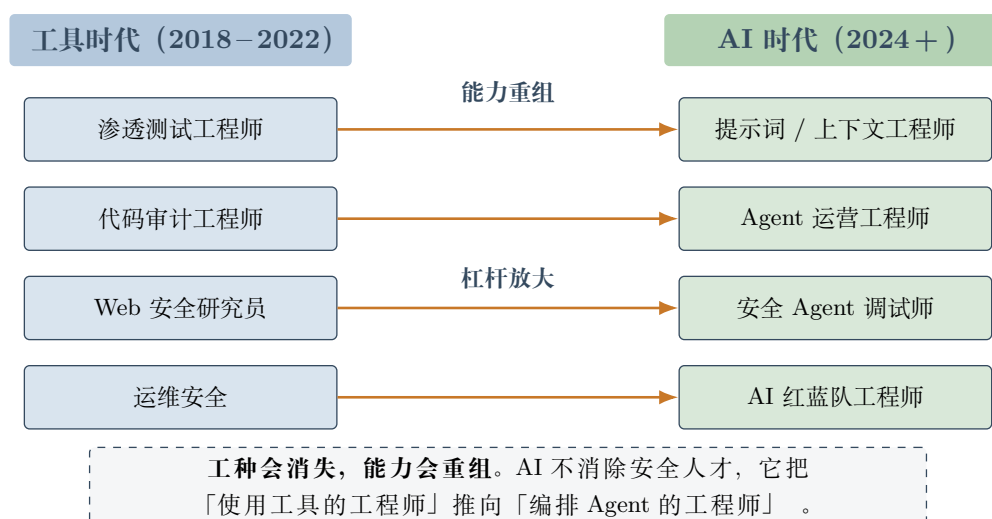


图 11: 人机协同关系迁移：工具时代 → AI 时代的能力边界平移。渗透测试工程师转向提示词工程师 / Agent 运营工程师，工种消失但能力重组，人并未消失。

人机协同不是减法，而是能力组合的重新洗牌。

李鑫的核心判断可以压缩为一句话：工种会消失，人不会消失；能力会被重组，而不是被消灭。把 AI 当作「替代者」来焦虑，不如把它当作「新生产工具」来重新设计自己的角色。渗透测试工程师转向提示词工程师 / Agent 运营工程师，不是降维，而是同一群人在新工具上的能力再分配。

4.6 过渡到下一章：从能力图谱到实践清单

把本章五位嘉宾的论述拼在一起，可以得到一条清晰的行动线：

1. 先确认自己的**技术深度**（支点）是否足够——这是杠杆生效的前提；
2. 再评估自己对**模型与 Agent 的掌控力**——知道边界才能踩浪而行；
3. 然后找到与大厂错位的 **15 度夹角**——避免被模型升级直接清扫；
4. 最后接受**人机协同关系的迁移**——工种会变，但人不会消失。

这条行动线的下一步，就是把「认知」落地为「工程」。下一章将沿着洛熙、宁大叔、蔡仪给出的五条具体建议展开：知识工程怎么做、长链路怎么验证、上下文与记忆怎么管理、基模与框架怎么适配、Claude Code 怎么实践——从能力图谱走进可执行的实践清单。

4.7 本章小结

- **AI 是杠杆，不是替代者**：技术深度是支点，AI 是杠杆，产出效率可放大 10-20 倍；没有支点的人拿到杠杆也撬不动。
- **重复化、80% 验收即可的工种最危险**：标准化测试、规则运营等流程固定的工作替代路径清晰；多链路组合、跨域绕过、首次发现型漏洞仍是人类领地。
- **掌控力是新的核心竞争力**：对所选模型与 Agent 架构的边界、长短板要有完整认知；Skills、Roles、Hooks 等基础概念的区分是检验掌控力的试金石。
- **15 度夹角是务实出路**：个人无法与头部公司硬碰，应保留人类长处、借模型能力扩展自身；夹角领域通常是大厂主流投入但回报曲线尚未陡峭的方向。
- **人机协同是关系迁移，不是减法**：工种会消失，能力会重组，人不会消失；渗透测试工程师将转向提示词工程师 / Agent 运营工程师，省下的时间会被新需求填满。
- **从认知到行动的过渡**：本章的能力图谱将在下一章转化为可执行的工程清单——知识工程、验证回路、记忆管理、基模适配、Claude Code 实践。

5 给智能攻防团队的实践建议：知识工程、记忆、Agent 框架与 Claude Code

圆桌进行到 01:06:00 之后，话题从「能力雷达」与「人才图谱」收束到一个更具体的工程问题：下一步该把团队的有限资源投到哪里。洛熙、宁大叔、蔡仪三位嘉宾从各自的工程视角给出了五条以上的具体建议——它们彼此呼应，最终拼成了一张面向智能攻防团队的实践清单。本章按「知识工程 → 长链路验证 → 上下文与记忆 → 基模与框架 → Claude Code 实践 → 前瞻视角」的顺序展开，并尽量把每一条都落到可以照着执行的步骤上。

5.1 差异化的真正源头：把团队私有知识做成 AI 可消费的工程

洛熙在第一句就把问题抛得很硬：不要去卷模型，也不要去卷数据。模型的质量、训练数据的体量都是模型厂商在自然迭代中会自己拿走的红利；安全团队若把精力花在那条主航道上，无异于「替操作系统厂商做防御补丁」——做得再好也不过是在替别人补盲区。真正属于团队、且不会被下一次模型升级清扫的，是**专属于团队的安全知识工程**：把多年积累的漏洞模式、攻击链拓扑、内部红队沉淀的判断规则，转写为 AI 可以稳定消费的形态。

这条结论的另一面是 RAG 的反思。洛熙明确指出：以 RAG（即向量召回）为代表的「向量召回 + 拼回上下文」的范式，正在被业界悄悄抛弃。原因不是 RAG 本身做错了什么，而是**向量空间的相似度并不等于人类语义的相关性**——把两段文档的 embedding 拉近，并不能保证它们在「攻击意图」「漏洞类型」「修复路径」这些人类关心的维度上也彼此对齐。结果是上下文被语义噪声污染，模型反而被一些“看起来很像、其实毫不相关”的片段带偏。他举了个正在落地的方向作为对照：南美团队在做英国图风格的图谱（也就是常说的“鸚鵡头”⁴），用图结构把节点间的关系显式编码进来，从而保证召回出的上下文是“语义相邻”而不是“向量相邻”。

⁴这里指业内对 Knowledge Graph 工程实现风格的口语化代称，强调“以节点 + 关系边”对齐人类语义，而非以向量相似度对齐。

向量 RAG 的语义对齐 vs 图结构的关系对齐

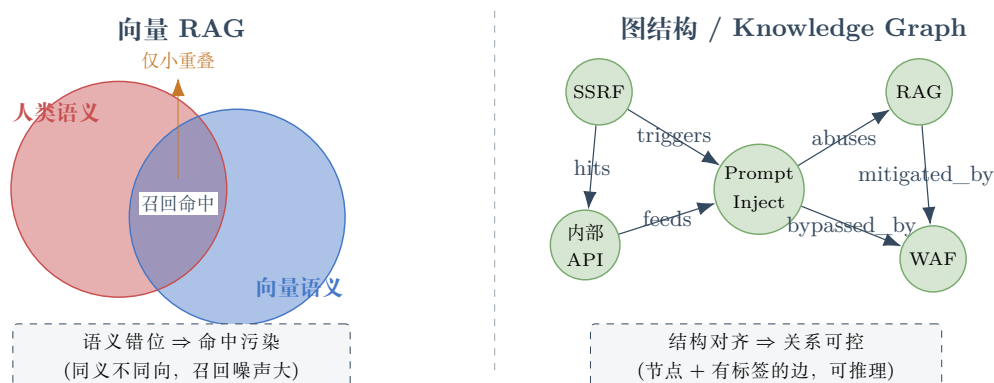


图 12: 向量 RAG 的”语义球错位”与图结构的”显式关系对齐”。左侧：query 与文档在向量空间近似，但在人类语义上其实无关，导致上下文被无效片段污染；右侧：图结构通过显式的关系边把”漏洞类型 → 修复模式 → 历史案例”链起来，召回的是关系上相邻的节点。

差异化竞争点不在卷模型/卷数据，而在专属知识工程。模型厂商会持续替你升级”通用智能”，但他们不会替你沉淀”你这个团队、你这个行业才知道的攻击直觉”。把这部分知识图谱化、可被 Agent 稳定调用，才是不会被下一代模型一夜清扫的护城河。

洛熙顺势补了一个更前瞻的判断：未来很多工具不再是给人用的，是给 AI 用的。他用自动驾驶作类比——最理想的自动驾驶不是让车在我们既有的、混乱不稳定的高速公路上学会驾驶，而是直接给自动驾驶汽车专门修一条自动驾驶专用路。同样的逻辑迁移到安全工具栈：与其让 Agent 去理解一个为人类设计的、UI 和文案都写给人看的扫描器界面，不如把扫描器本身改造成”以 Agent 为一等公民”的接口形态——稳定、结构化、可被推理的输入输出。这条判断会直接影响接下来两三年内部工具的选型与重写。

未来很多工具不再是给人用的，是给 AI 用的——造一条自动驾驶专用路。与其让 AI 在一堆为人类设计的不稳定接口里”硬开车”，不如把接口本身改造成 Agent-native：稳定的协议、结构化的回包、明确的副作用边界。这不是给 AI 让步，而是把工程负担放到正确的一侧。

5.2 长链路要有验证 + 反馈机制

洛熙在第二节给出的是一条工程铁律：安全任务普遍是 ABCD 一层一层递进的长链路——上一步的中间结论是下一步的输入，一旦某一步偏了，后面的所有路径都会跟着偏。而模型厂商目前的训练大量集中在常见路径（常见思考路径）上，安全攻击链恰恰是那种”非常路”——同一个目标可能要经过若干非典型跳转才能命中。结果是：模型

在长链路中段稍有飘移，自身却毫无察觉，因为它”没有这些知识，不知道这里是错误的”。

应对方法不是去训一个更聪明的模型，而是在 **Agent 工程外侧**加一层”实时观察 + 知识补充 + 引导回正”的反馈回路：

- **设阶段验证**：把长链路切成若干阶段，每一阶段输出都有一个可程序化的 check（不是再叫一次 LLM 自评，而是用规则、签名或工具实测）。
- **设回调点**：当某阶段验证失败时，不要让链路继续往下推，而是回调到上一稳定状态，并把错误信号作为新的上下文注入。
- **设安全护栏**：对于那些”一步走错就无法回头”的动作（比如发起真实请求、写文件、清理痕迹），必须有强制的人工或规则护栏，避免 Agent 把一个错误推理直接落地为不可逆操作。
- **设知识补丁**：观察到 Agent 在某类节点反复出错时，把”该领域的正确知识点”显式补进知识工程（参见上一节），让下一次同类任务从根上不再走偏。

洛熙在这里顺手给”智能”下了一个相当朴素的定义：**智能 = 已知的知识 + 已做的实践经验之间的连接**——也就是另一种”知行合一”。AI 推出来的路径未必是我们想要的路径，所以必须有外部验证机制把”AI 认为对”和”工程上确实对”这两件事拉齐。

5.3 上下文与记忆管理：工程主线，而不是配角

第三条建议是最容易被低估的一条：**记忆管理要大于工具调用**。很多团队把精力放在”接入更多 MCP / 加载更多 SKL”，但 Agent 真正卡住的地方往往不是”工具不够”，而是”记得不对”。

洛熙在这里给出了一个非常具体的工程观察。当下主流模型号称的上下文窗口（他用了”8M”这种粗略口径⁵），并不意味着你真的可以塞 8M token 进去然后期望模型对中段也保持注意力——**占用率一旦到达 60%–70%，模型就开始出现明显漂移**：

- **首尾偏置**：底层注意力机制对头部和尾部的 token 更敏感，中段在”开放态”，最容易被忽略或误读。
- **压缩有损**：当窗口接近极限时常见的策略是”压缩历史”，但根据信息论，每一次压缩都会丢失一部分知识——压缩不是免费的。
- **召回不可绕开**：所以记忆机制不仅要能”装下”，还要能”按需取回”——把记忆当成知识库去管理，需要设计召回触发条件、召回粒度、召回置信度。

⁵字幕识别为”EM/8M”等，按嘉宾原话理解为”数 M 级”窗口；具体数字以使用模型时官方公告为准。这里”8M（带引号）”是嘉宾的措辞习惯，强调”标称值不等于有效值”。

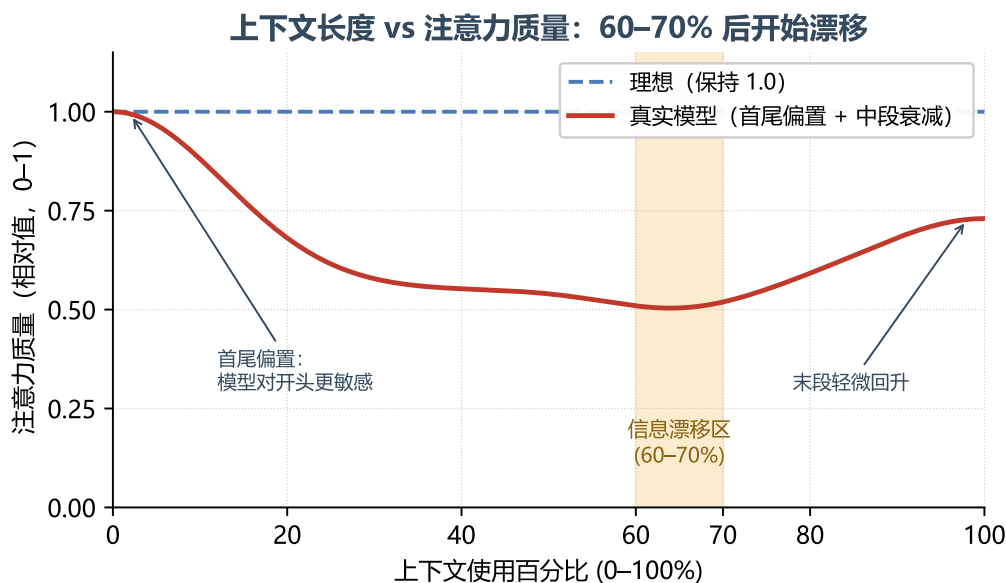


图 13: 上下文长度 vs 注意力质量的衰减曲线示意。横轴为窗口占用率，纵轴为模型对中段 token 的有效注意力。曲线在 60%–70% 处出现明显拐点，且头/尾 token 始终高于中段 token（首尾偏置）。这条曲线是”什么时候要触发记忆压缩”的工程依据。

为了让”记忆”这件事更可执行，下面把 Claude Code 体系里几个常被混淆的概念清单化——它们各自承担不同的”记忆/工具/规则”角色：

Claude Code 概念速查清单

- **Skills (技能):** 可被 Agent 主动加载的”做事方法包”，定义”在某类任务下应该按什么流程、调什么工具去解决”。回答的是”怎么做”。
- **Roles (角色):** 约束 Agent 的人格/语气/职责边界，决定它”以什么身份回答、不该越界做什么”。回答的是”我是谁”。
- **Hooks (钩子):** 在生命周期事件（如工具调用前后、会话结束）由 harness 自动触发的脚本/规则，是真正能”自动化执行”的地方。回答的是”什么时候自动做什么”。
- **Subagents (子代理):** 把一个独立子任务派发给具有独立上下文的下级 Agent，主 Agent 只看回包，是给主上下文”减负”的关键手段。回答的是”谁来做”。

使用提醒: 把”自动行为”放进 Hooks（harness 才执行得了），把”做事流程”放进 Skills，把”身份/语气”放进 Roles，把”独立子任务”派给 Subagents——四者各司其职，不能拿其中一个去当另一个用。

5.4 基模与框架的契约：没有万能 Agent 框架

第四条是洛熙花了相当时间反复强调的一条：“不存在万能的框架”。当下大家都在谈 Agent 工程要怎么做、Agent 框架要怎么做，但很多人忽略了一件事：**框架只是壳，基础模型才是驱动核心**。换句话说，Agent 框架与基模之间需要形成一份“契约”——框架的设计要为基模的长短板做适配，而不是反过来期望“我做一个万能框架，可以适配任何模型”。

这条结论自然地引出对 MCP / SKL / 多智能体协作的判断：

MCP / SKL / 多智能体协作只是过渡产物。它们当下确实在「补模型的领域知识、补外部工具能力、补单 Agent 上下文不够用」这三件事，但它们的核心价值会随基模迭代被逐步内化——别把这种过渡形态的脚手架，误当作长期答案去重金投入。把这条警告反过来读就是：**要押注那些基模迭代之后仍然有价值的东西**（知识工程、记忆管理、验证回路），而不是押注“今天的工具协议形态”。

把上面四条建议拼起来，安全 Agent 的工程栈就成了一张五层图：底层是基础模型，往上是上下文/记忆，再往上是知识工程（图谱），再往上是工具与 MCP/Skills 适配层，最顶层是验证与回路。每一层都有自己的工程主张，缺一不可。



图 14: 安全 Agent 工程栈。从下到上：基模（驱动核心）→ 上下文/记忆（占用率与压缩管理）→ 知识工程（图谱化的私有领域知识）→ 工具与 MCP/Skills（Agent-native 接口适配）→ 验证与回路（阶段 check、回调点、安全护栏）。建议把投入按“自下而上仍然有价值”的顺序往中下层倾斜。

洛熙也给出了一个很务实的“先单后多”路径：先把单 Agent 做扎实，确保它的整条路径是可验证、可回放、可调试的；只有当任务确实复杂到一个 Agent 装不下时，再考虑分布式的多 Agent。**多 Agent 不是默认起点，而是单 Agent 容量耗尽之后的扩展**

手段——这一点和当下”言必称多智能体”的风气形成鲜明对照。

5.5 Claude Code 实践指南：不要把”读文档”也外包给 AI

宁大叔在这里接过话筒，给出的角度更贴近”日常工程师怎么用好 Claude Code”。他先给出一个判断：在他看到的本届比赛实践里，**Claude Code (CC) + OPUS** 系列已经是市面上完整度最高的 Agent 形态产品之一，几乎可以承接”与模型交互”的全部任务，做 MVP 非常合适。但与此同时，他观察到一个普遍的隐患：大家越是依赖 *CC + OPUS* 写代码，就越是在丧失对这个 *Agent* 本身的掌控力。最常见的两种”反模式”是：

- 完全依赖 AI——把 Agent 当黑盒，写出问题再让 AI 自己改；
- 把文档链接丢给 AI 让它自己读——这正好踩中洛熙在前面说的”AI 会有发散和关注点漂移”的问题，原本文档里就已经写好的功能，被 AI 漏读之后我们又花精力”重新实现”。

宁大叔的反例是 Skills 与 Roles 这两个概念：有多少团队真的能一句话说清楚 Roles 和 Skills 的区别？又有多少人系统读过 Hooks 文档、知道哪些自动化只能由 Hooks 完成而不能由 prompt 模拟？文档没读透，”手搓框架”的本领就被悄悄掏空了。

落到执行层面，可以按下面四步推进：

1. **先逐行读官方文档。** Skills、Roles、Hooks、Subagents 各读一遍，自己写一段简短的”概念边界笔记”——不要让 AI 替你读。
2. **自己手搓一个最小 Agent。** 不依赖 CC，先用一个最小的 loop 把 prompt、工具调用、记忆压缩、验证回路从零跑通一遍，理解每一处的副作用。
3. **再用 Claude Code 做工程化补完。** 当你已经知道每一层”应该做什么”之后，再用 CC 的现成能力把工程化补齐——你这时候用 CC 是”知情同意”，不是被动接受。
4. **保留对 prompt/记忆的全链路控制。** 任何让你”看不到 prompt 是怎么拼的、记忆是怎么压缩的”的封装，都要审慎——MVP 阶段可以接受，长期产品化必须能回到底。

宁大叔最后用一句话收束：”手搓框架”这个本领，是当下最不可替代的本领。它不是要否定 Claude Code 的价值，而是要保住”我们仍然知道这一切是怎么工作的”那条底线——一旦把读文档、写框架、调记忆、改 prompt 全外包给 AI，我们就只能在 AI 能力的边界内打转，再也走不到它够不到的地方。

5.6 蔡仪补充：保持前瞻视角，走在 AI 公司前一点

蔡仪在这一段没有再展开新工程主张，她更想补一个“姿势”上的提醒：**不要和 AI 公司” 并排走”**。原因很现实——如果你的工作主航道恰好和 AI 公司的下一次发版高度重叠，那你今天用三个月做出来的能力，可能一次模型升级就被抹平。要避免被这种”一步超过去”打懵，团队应该把目光往外推一点：

- **预判下一步发力点：**不是被动等 AI 公司发布、再决定要不要跟，而是提前判断”未来这一类问题会成为新的瓶颈”，然后把团队的资产先布到那里。
- **用行业老人的” 前瞻视角” 换护城河：**AI 公司很强，但他们对某个垂直行业的真实痛点、合规边界、客户认知，不可能比一个干了十几年的安全老兵更敏感——这是行业老人最后、也是最稳的护城河。
- **走在 AI 公司前面” 一点点” 就够：**不需要遥遥领先，只要你的方向盘比他们早转一格，就不会被反超时直接出局。

把蔡仪这段和洛熙的” 知识工程”、宁大叔的” 手搓框架” 放在一起读，会发现三者其实是同一句话的三种表述：把价值押在那些 AI 公司不会顺手替你做掉的地方。

5.7 拓展阅读

本章涉及的两个外部资源，建议读者对照阅读：

- **Anthropic Claude Code 官方文档：**Skills、Roles、Hooks、Subagents 的权威定义与最新行为说明。逐行读，不要让 AI 替读——参见 Anthropic 官方文档（建议从 Claude Code 项目首页进入）。
- **Anthropic Petri 项目：**第 3 章提到的” 模型对自身行为的自动化评估/审计” 项目，与本章”AI 难以自评测试完备性” 的判断互为参照。具体接口与样例参见 Anthropic 官方文档。

5.8 本章小结

- **差异化在知识工程，不在卷模型：**把团队私有的安全知识图谱化、Agent-native 化，是会被下一代模型清扫掉的护城河。
- **RAG 的污染来自” 向量语义 $\neq$ 人类语义”：**图结构（鸮头/英国图风格）正在替代直接的向量召回，是更可靠的知识接入方式。
- **长链路要配套验证回路：**阶段验证、回调点、安全护栏、知识补丁，缺一不可——长链路里”AI 自己不知道自己错了” 是常态。

- **记忆 > 工具**：标称的”8M（带引号）”窗口在 60%–70% 占用后就开始漂移，首尾偏置和有损压缩是必须正视的工程现实。
- **基模与框架是契约关系**：没有万能 Agent 框架；MCP/SKL/多智能体只是过渡产物，不要重金押注脚手架本身。
- **Claude Code 是好 MVP，但不要把读文档外包**：先读 Skills/Roles/Hooks，再手搓最小 Agent，再用 CC 补完，最后保留 prompt/记忆全链路掌控。
- **保持前瞻视角**：与 AI 公司并排走会被一步超过；行业老人的最后护城河是判断力——把资产布在 AI 公司不会顺手替你做掉的地方。

6 总结与延伸

圆桌的最后十几分钟，三位嘉宾把话题从「比赛、工程、组织」逐步收束到「人本身」。这一章先把闭幕段的实质讨论整理出来，再以作者视角对全片做一次结构化回顾，并把前五章的散点收束为一张可以在合上笔记之后继续生效的认知地图。

6.1 蔡仪的临别提醒：保持前瞻视角的接力

在洛熙与宁大叔围绕 Claude Code、知识工程、手搓框架展开充分讨论之后，蔡仪补充了一段从行业视角出发的「速度差」提醒：

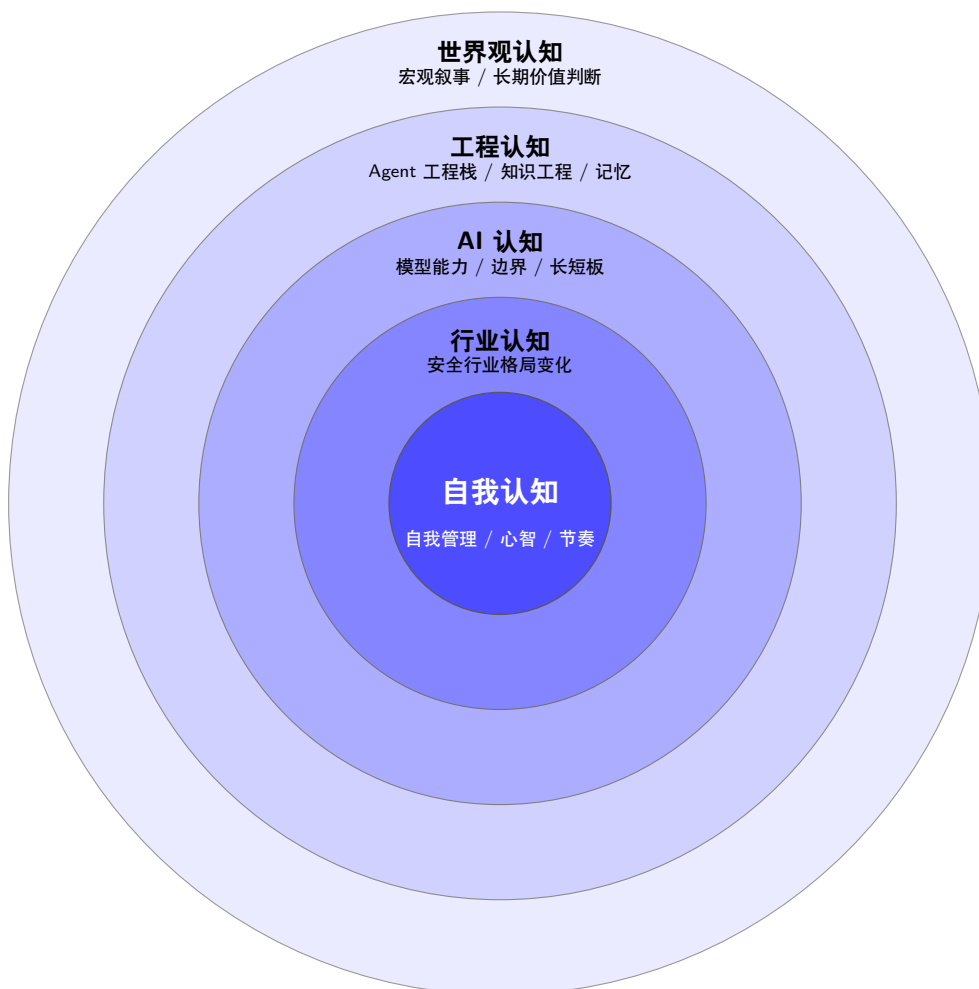
- 现在很多团队的关注点其实需要稍微挪一挪：不要总是和 AI 公司并排走，因为只要对方一次小升级，并行者就会被一步超过。
- 行业老兵的优势在于前瞻视角——靠多年沉淀去预判 AI 的下一发力点，把目光放得比当前模型的能力曲线再远一点。
- 与其追平，不如走在 AI 公司前面一点点：在他们尚未铺到的场景、尚未抽象的工程问题上提前卡位。

这段提醒接续了第 5 章「建议六」的脉络，但站位更高：那里讨论的是「围绕基模做工程」的微观策略，这里强调的是「围绕 AI 公司的发力路径做战略选择」。两者合起来，构成个人/小团队的双层护城河。

6.2 李鑫的总结：构建强认知系统才是真正的护甲

李鑫的收束更靠近「人」这一侧：三位嘉宾从战况、工程、行业三个不同视角描述同一件事，拼起来才是 AI 时代的完整图景；而对个体来说，最关键的资产是认知系统。

认知系统五维同心圆：构建 AI 时代的强认知架构



「认知系统抵御焦虑，决定你能不能在 AI 浪潮里继续踩浪」

——李鑫收尾观点的合成转写

图 15: 认知系统五维同心圆：自我 / 行业 / AI / 工程 / 世界观。

李鑫给出的论据有两层：

- **AI 的弱点是感知**：模型再强，对真实世界的现场感、跨域整合、价值判断仍然偏弱。这是人类核心能力的所在，也是认知系统能够安放的空间。
- **冲击点是心智，不是技术**：过去一年，真正让人焦虑的不是 AI 把谁替代了，而是对自我、职业、行业、世界观的重新刷新。没有内核的人会被这股洪流推着走；有强认知系统的人才能把冲击翻译成机遇。

AI 时代真正的稀缺资产是「认知系统」。李鑫所定义的认知系统由五个同心圆组成：

1. 自我认知：知道自己长在哪、短在哪、判断从哪里来。
2. 行业认知：理解所在行业的演化节奏、利益结构、规则变迁。
3. AI 认知：分得清基础模型、Agent 框架、工具协议各自的边界与节奏。
4. 工程认知：把行业问题翻译成可被 AI 消费的工程化问题。
5. 世界观认知：在剧烈的价值冲击中保持稳定的内核。

技术能力强但认知系统空心，依然会错过新的机遇；反过来，认知系统稳，技术哪怕只有中位水平，也能借助 AI 杠杆放出 10-20 倍产出（联动第 4 章）。

反例示意：认知系统空心 → 焦虑放大 → 错失机遇。

- 只追工具，不追判断：每出一个新模型就重写一遍 Agent，结果永远在 0.x 版本之间打转。
- 把焦虑等同于成长：天天被 AI 推着冲，却没把任何一次冲击沉淀成行业洞察。
- 把替代焦虑外化为对行业的怀疑：拒绝接受新工种（提示词、Agent 运营），最后连原来的渗透岗位也转型不出去（联动第 4 章「人机协同迁移」）。

强认知系统不是更勤奋，而是更稳；它让人在洪流里不至于被卷成焦虑的副产品。

6.3 段斌的收尾与赛事预告

段斌作为主持人合上话筒前，给出了三件事：

- 今晚的视角定位：本场圆桌站在社区与行业的视角，看两届比赛带来的变化、嘉宾对未来的判断与分歧——这与第 1 章开场所给的「视角切换」前后呼应。
- 后续直播：还剩两晚直播，每天晚上 20:00，会继续邀请不同方向的嘉宾来谈 AI 安全。
- 线下决赛：4 月 25 日在北京举行线下总决赛，含答辩与线上交流环节，欢迎读者关注和参与。

把全片五大主题与本章「认知收束」串成一条逻辑链。

1. Ch1 战况实证：54 个 flag、前 10 晋级、RAG 攻击链与零界抱团——给出一切讨论的事实底座。

2. **Ch2 赛制争议**：扣分激进、先到先得、公共靶场——从规则层揭示 AI 时代「公平观」的更新。
3. **Ch3 能力评估**：AI 平权下限、覆盖率短板、灵感型漏洞依旧依赖人类——给出当前 AI 的能力坐标。
4. **Ch4 人才图谱**：技术深度作支点、AI 作杠杆、掌控力是核心、人机协同迁移而非替代。
5. **Ch5 工程清单**：知识工程、记忆/上下文、Agent 框架与基模契约、Claude Code 学习路径。
6. **Ch6 认知收束**：把上面五层从外向内压缩为一个面向个体的认知系统，作为读完本笔记后能持续工作的「内核」。

后续追踪建议：关注 4 月 25 日北京线下决赛的复盘材料、剩余两晚 20:00 直播的嘉宾名单，以及主办方后续是否公开主赛场对话日志（这与 Ch1 中提到的「赛后公开」承诺相对应）。

6.4 作者的结构性回顾：把全片压成五张卡

下面是作者基于全片做的浓缩回顾，每章只保留最核心的两行论点，便于快速翻查。

- **第 1 章·战况实证**：本届智能渗透赛已经能给出可量化的实证；RAG 攻击链与零界抱团是最具教学价值的两个现象。
- **第 2 章·赛制争议**：激进扣分与先到先得不是「不公平」，而是把 AI 时代红队的真实分布折叠进了赛制设计。
- **第 3 章·能力评估**：AI 已能拉齐渗透下限、放大上限，但在覆盖率自评与「范式未提出前的灵感」上仍是明显短板。
- **第 4 章·人才图谱**：技术深度不会贬值，反而成为杠杆支点；新的核心竞争力是掌控力与 15 度夹角的方向选择。
- **第 5 章·工程清单**：差异化在专属知识工程、记忆/上下文管理、基模契约，以及把 Claude Code 当作可逐行精读的最佳实践样本。

6.5 跨章交叉的关键张力

合在一起看，全片有一条贯穿性的「张力链」，读者如果没建立认知系统，单章读起来会觉得彼此打架；有了认知系统之后，会看到它们其实是一根线：

- **扣分激进 (Ch2) → AI 学习速度 (Ch3)**: 因为 A 队的解法被 B 队复用得太快, 所以必须用更陡的扣分曲线维持区分度。
- **AI 学习速度 (Ch3) → 杠杆模型 (Ch4)**: 既然下限被拉齐, 差距就只能体现在「深度 × AI」这条放大曲线上。
- **杠杆模型 (Ch4) → 知识工程 (Ch5)**: 要把杠杆撬得动, 私有知识必须先工程化、图谱化, 否则杠杆只是空转。
- **知识工程 (Ch5) → 认知系统 (Ch6)**: 团队的知识工程化是「外脑」, 个人的认知系统是「内核」; 外脑离开内核就成了无主之物——这就是为什么全篇必须以认知收束。

换句话说: 扣分激进是表象, 认知系统才是底层。中间的能力评估、人才图谱、工程清单是把这两端连起来的传动轴。

6.6 给读者的下一步

把笔记合上之后, 作者建议至少做以下 3-5 件事中的一件:

- **选 1 个基模深度啃文档**: 像第 5 章所述那样, 把 Claude Code 的 Skills、Roles、Hooks 文档逐行读一遍, 不要把读文档本身也外包给 AI。
- **把团队私有知识图谱化**: 从最常用的 5-10 类安全知识开始, 先小规模做出图结构样例, 再决定要不要替换现有向量 RAG (呼应 Ch5 的 RAG 污染问题)。
- **为自己画一张五维认知图**: 对照本章的同心圆, 列出自我 / 行业 / AI / 工程 / 世界观五个维度上当前的弱项, 每月重检一次。
- **跟踪 4 月 25 日线下决赛复盘**: 把决赛公开的对话日志与本笔记 Ch1、Ch2 的现象做对照, 看哪些机制经过线下放大后是否仍然成立。
- **在团队内部跑一次「15 度夹角」练习**: 把团队当前的主航道列出来, 再标记与大厂主航道相比偏出的那 15 度, 作为下一年的发力方向 (呼应 Ch4)。

6.7 拓展阅读 (可选)

- Anthropic 关于 Claude Code 的公开文档: Skills / Roles / Hooks 三类机制是当前最系统的 Agent 形态参考。
- 关于上下文衰减与首尾偏置的工程实证: 可结合 Ch5 的「数 M 级」上下文窗口讨论一起读。

- 图结构知识工程（Graph + 知识工程）相关综述：用以替代向量 RAG 的中长期方向。
- 公开复盘材料：4 月 25 日北京线下决赛、剩余两晚 20:00 嘉宾分享。

6.8 本章小结

- 蔡仪的临别提醒：不要与 AI 公司并排走，要靠行业沉淀走在它前面一点，把握下一发力点。
- 李鑫的总结：AI 对个体的冲击主要在心智层，强认知系统（自我 / 行业 / AI / 工程 / 世界观）才是真正的护甲。
- 段斌的收尾：本场是社区与行业视角的圆桌；剩两晚 20:00 直播，4 月 25 日北京线下决赛。
- 全片可以收束为一条链：战况实证 → 赛制争议 → 能力评估 → 人才图谱 → 工程清单 → 认知收束。
- 中间几章的关键张力（激进扣分、AI 学习速度、杠杆模型、知识工程）必须在「认知系统」这一层才能闭合。
- 给读者的下一步：深啃一个基模、做团队知识图谱、画自己的多维认知图、跟踪线下决赛。
- 警惕反模式：认知空心 → 焦虑放大 → 即便技术能力强也会错过 AI 时代的新机遇。